

DotTech Software

dottechsoftware.lk

Internship Test Tasks

Choose one track based on your interest and submit within 3–5 days. You can do more than 1 task and take days accordingly.

 Frontend Build a responsive product listings UI with search and filtering.	 Backend Build a clean REST API for a task manager with validation and error handling.
 QA & Testing Write a test plan, test cases, and bug reports for a given feature spec.	 UI/UX Design Design a polished product listing UI and detail screen in Figma.

How to Submit

- Fill in the Candidate Questions section at the end of this document before submitting.
- Choose ONE task that best matches your interest (note your 1st and 2nd choice in the questions section).
- Complete the task within 3–5 days of receiving this document.
- Submit your work by email to:
 - Frontend, Backend & UI/UX: share a GitHub repo or Figma link (invite dottechsoftwares@gmail.com)
 - QA: share a GitHub repo link or attach your document
- Your README or document should clearly explain how to run / review your submission.

Task 01 Frontend Development

🕒 3 – 5 days Your choice of tech

What we value: Clean work over impressive-sounding work. Thoughtful decisions over feature overload. It's okay to not finish every bonus item — focus on doing the core requirements well.

SCENARIO

You have been asked to build a product listings page for an e-commerce-style web application. The goal is to display a set of items in a clean, browsable format with search and filtering.

REQUIREMENTS

Build a web page that fulfils the following:

- Display a grid/list of at least 8 product cards using hardcoded mock data.
 - Each card must show: product name, category, price, a short description, and an image (placeholder is fine).
- Add a search bar that filters products by name in real time.
- Add a category filter (e.g. dropdown, tabs, or chips) — at least 3 categories.
- When no results match, show a clear "No results found" state.
- The layout must be fully responsive (works well on mobile and desktop).
- Code must be structured and readable — use components/functions, avoid everything in one file.

- ★ **Bonus:** Add a "Sort by" option (price low→high, high→low, name A→Z).
- ★ **Bonus:** Add a loading skeleton or animation before cards appear.
- ★ **Bonus:** Add a light/dark mode toggle.

TECH

Use any technology you are comfortable with.
Examples: React, Vue, plain HTML/CSS/JavaScript, Svelte — all acceptable.
If using a framework, you may use any component library or build your own styles.

DELIVERABLES

- A public GitHub repository (or invite dottechsoftwares@gmail.com as a collaborator).
- A README.md with: brief description, how to run locally, and any decisions you made.

EVALUATION CRITERIA

Criterion	What we look for
UI Quality	Cards are well-designed, readable, and feel polished
Functionality	Search, filter, and empty state all work correctly
Responsiveness	Layout adapts cleanly at mobile and desktop widths

Code Structure	Logical component breakdown, readable variable names
Code Cleanliness	No dead code, no console.logs, consistent formatting
README	Clear enough for a stranger to run in 2 minutes

Task 02 Backend Development

 3 – 5 days Your choice of tech

SCENARIO

You have been asked to build the backend API for a simple task-management feature. Another developer will build a frontend on top of your API, so it needs to be clean, consistent, and well-documented.

THE DATA MODEL

Each Task object should have:

- id — unique identifier (auto-generated)
- title — string, required
- description — string, optional
- status — one of: "todo" | "in_progress" | "done"
- createdAt — timestamp, set automatically on creation

REQUIREMENTS

Build a REST API that exposes:

- GET /tasks — return all tasks
- GET /tasks/:id — return a single task by ID
- POST /tasks — create a new task
- PUT /tasks/:id — update a task
- DELETE /tasks/:id — delete a task

All endpoints must also:

- Validate input — return a clear error if required fields are missing or invalid.
- Return consistent JSON (e.g. { data, error }) with correct HTTP status codes.
- Use any storage: in-memory array, JSON file, or a real database — your choice.
- ★ **Bonus:** Add pagination to GET /tasks (e.g. ?page=1&limit=10).
- ★ **Bonus:** Add a filter query param (e.g. ?status=in_progress).
- ★ **Bonus:** Add JWT authentication — protect all endpoints except POST /auth/register and POST /auth/login.

TECH

Use any backend language or framework you are comfortable with.
Examples: Node.js/Express, Python/FastAPI, Go — all acceptable.
If you use a database, include a Docker Compose file or clear setup instructions.

DELIVERABLES

- A public GitHub repository (or invite dottechsoftwares@gmail.com).
- A README.md with: how to run, all endpoints with example curl commands, and trade-offs made.

EVALUATION CRITERIA

Criterion	What we look for
API Design	Endpoints follow REST conventions, sensible URLs and methods
Validation & Errors	Returns the right status codes and clear error messages
Code Structure	Logic separated from routes, readable and organised
Response Consistency	All endpoints return a predictable JSON shape
README / Docs	Another developer can run and use the API in under 5 minutes
Bonus (optional)	Auth, pagination, or filtering implemented correctly

Task 03

QA & Software Testing



3 – 5 days

No coding required · bonus: automation

SCENARIO

You have been handed the specification for two features on a web application. Your job is to make sure these features are ready to ship — write a test plan, detailed test cases, and three sample bug reports.

FEATURE SPECIFICATION

Feature 1 — User Login

- User enters email and password on the login page.
- On success: redirected to the dashboard. On failure: error message shown below the form.
- After 3 consecutive failed attempts: account locked for 5 minutes, message shown.
- Email must be valid format. Password must be at least 8 characters.
- "Forgot Password" link navigates to the password-reset page.

Feature 2 — Product Search

- Search bar present at the top of the product listing page.
- Search fires automatically after the user stops typing for 500 ms (debounced).
- Results show: product name, price, and category.
- If no products match, show: "No results found for [term]".
- Search is case-insensitive and trims leading/trailing spaces.
- Clearing the search field restores the full product list.

REQUIREMENTS

Part A — Test Plan (1–2 pages):

- Scope, out of scope, test types, and any assumptions.

Part B — Test Cases (minimum 15):

- Cover both features. Include functional, negative, edge case, and boundary scenarios.

Each test case must include:

- Test Case ID · Title · Preconditions · Steps to Reproduce · Expected Result · Priority

Part C — Bug Reports (exactly 3):

Write 3 bug reports for bugs you could imagine finding. Each must include:

- Bug ID & Title · Environment · Steps to Reproduce · Expected / Actual Result · Severity · Attachments (describe)

- ★ **Bonus:** Write automated tests using any framework (Cypress, Playwright, Jest, Selenium, etc.).
- ★ **Bonus:** Create a test matrix showing coverage across browsers or screen sizes.

DELIVERABLES

- A PDF, Word, or Markdown document with your Test Plan, Test Cases, and Bug Reports.
- If automated tests: include them in a GitHub repo with run instructions.

EVALUATION CRITERIA

Criterion	What we look for
Coverage	Test cases cover happy path, negative, edge, and boundary scenarios
Clarity	Steps are specific enough for anyone to reproduce
Bug Report Quality	Detailed, reproducible, and correctly prioritised
Test Plan	Scope and approach are clearly articulated
Structure	Document is well-organised and easy to navigate
Bonus (optional)	Automated tests run correctly and cover meaningful scenarios

Task 04 UI/UX Design



3 – 5 days

Figma required

SCENARIO

You have been asked to design the UI for a product listing feature of an e-commerce web application. The design needs to be polished, user-friendly, and ready to hand off to a developer. Focus on clean visual hierarchy, intuitive navigation, and a consistent design language.

SCREENS TO DESIGN

Design the following screens (web desktop + mobile breakpoints for each):

- Screen 1 — Product Listing Page
 - Product grid with cards (image, name, category, price, short description)
 - Search bar and category filter (tabs, chips, or sidebar)
 - Empty state when no products match the search
- Screen 2 — Product Detail Page
 - Large product image(s), full description, price, and an "Add to Cart" or "Enquire" CTA
 - Breadcrumb or back navigation
 - A related products section at the bottom
- ★ **Bonus:** Design a simple loading skeleton state for the listing page.
- ★ **Bonus:** Design a dark mode variant of both screens.
- ★ **Bonus:** Add a Figma prototype with clickable navigation between the two screens.
- ★ **Bonus:** Include a mini design system: color palette, typography scale, button and input components.

DESIGN GUIDELINES

You are free to choose your own color palette, typography, and style direction.
Use only free fonts (Google Fonts) and free icon sets (Heroicons, Feather, Material Icons, etc.).
Design for a clean, modern web application — not a mobile app (though mobile breakpoints are required).
Figma is required. You may use any Figma plugins or community resources you like.

DELIVERABLES

- A Figma file shared with view access to dottechsoftwares@gmail.com.
 - Or: export all frames as PDF and attach to your submission email.
- A short written note (in the Figma file or a separate document) covering:
 - Your design decisions — why you chose the layout, colors, and typography
 - Any trade-offs or things you would improve with more time
 - How you approached the mobile vs desktop breakpoints

EVALUATION CRITERIA

Criterion	What we look for
Visual Quality	Design feels polished, modern, and consistent
Usability	Layout is intuitive — key actions are easy to find and use
Component Consistency	Colors, spacing, and typography are applied consistently throughout
Responsiveness	Desktop and mobile breakpoints are both considered and well-executed
Figma Craft	Frames, layers, and components are named and organised clearly
Design Notes	Decisions are clearly explained and show genuine thought
Bonus (optional)	Prototype, dark mode, or design system increases the score

Candidate Questions

Please fill this in and include it with your submission.

Your Details

Full Name: _____

Email: _____

University / Institution: _____

Degree / Year of Study: _____

Q1 — Track Preference

Please indicate your 1st and 2nd choice of track. Tick the box (☐) for each.

1st Choice (the task you are submitting)

☐ Frontend	☐ Backend	☐ QA & Testing	☐ UI/UX Design
--------------------------------------	-------------------------------------	--	--

2nd Choice (the track you would explore next)

☐ Frontend	☐ Backend	☐ QA & Testing	☐ UI/UX Design
--------------------------------------	-------------------------------------	--	--

3rd Choice (the track you would explore next)

☐ Frontend	☐ Backend	☐ QA & Testing	☐ UI/UX Design
--------------------------------------	-------------------------------------	--	--

4th Choice (the track you would explore next)

☐ Frontend	☐ Backend	☐ QA & Testing	☐ UI/UX Design
--------------------------------------	-------------------------------------	--	--

Q2 — AI in Your Daily Life

How do you currently use AI tools in your day-to-day life? (e.g. ChatGPT, Gemini, Claude, Copilot, Midjourney, or any other tools.)

What kinds of tasks do you use them for, and how often?

Your answer:

Q3 — AI in Your Academic Work

How do you use AI tools to support your studies or academic projects?

Have you used AI for assignments, research, presentations, or learning new topics? Share a specific example if you can.

Your answer:

Q4 — AI in Your Coding Practice

How do you use AI tools when writing code?

For example: do you use GitHub Copilot, ChatGPT, or similar tools for autocomplete, debugging, explaining concepts, or generating code? What is your workflow?

Your answer:

Q5 — Your Take on AI in Software Development

In your own words: how do you think AI is changing the way software gets built? How do you see yourself adapting to that as a developer / designer / tester?

Your answer:

Questions?

Reach out to us at dottechsoftwares@gmail.com — we're happy to clarify any part of the task.

Good luck! We look forward to seeing what you build.

DotTech Software · dottechsoftware.lk